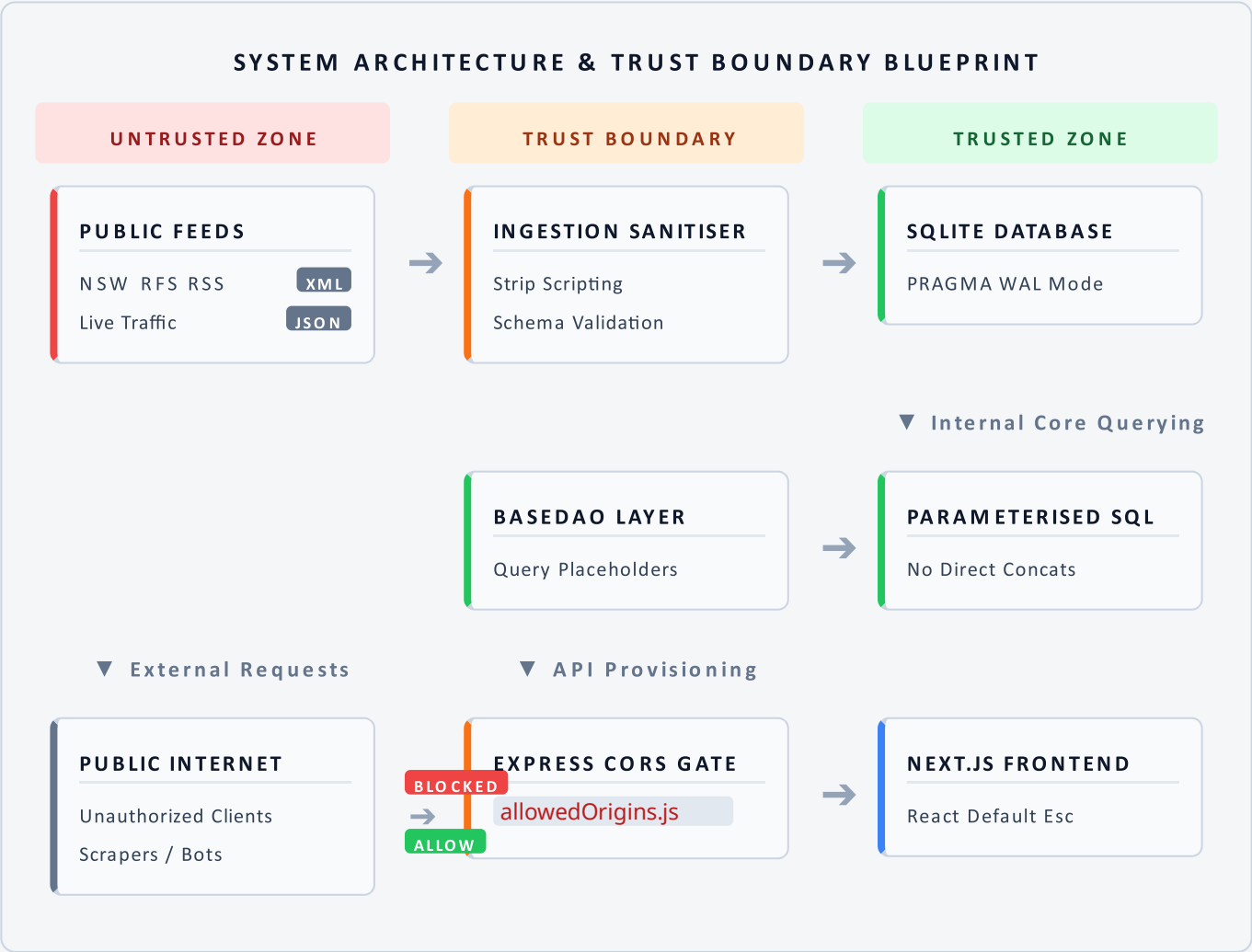


ROLE	Cybersecurity & Operations Lead (<i>Critical Signal</i>)
TEAM	Ettore Vescio, Christian Finocchiaro, Thomas Smith, Pradip Pandey
COURSE	ITC303/ITC306
ASSESSMENT	A3 — Solution Architecture & Working Prototype Walkthrough

PART 1: CYBERSECURITY EXECUTIVE SUMMARY

When architecting the Critical Signal Public Safety Alerts Aggregator & Explorer, we didn't view cybersecurity as a compliance checklist item we needed to check off before we could release, we considered it an integral part of the architecture itself. In our hybrid Agile approach, we introduced a NIST supported risk treatment workflow into the first few sprints.

Strategic Mandate: This proactive stance led us to define strict **trust boundaries** between the untrusted public feeds and our trusted application core, implementing automated security controls at every boundary transition.



PART 2: SLIDES, VISUALS, & SCRIPTS

Slide 1: Secure-by-Design Methodology & Trust Boundaries

Slide Visual Layout:

- Clean diagram showcasing NIST-aligned Risk Treatment cycle in Sprints.
- Visual representation of the **Trust Boundary** between public feeds (RFS, TfNSW) and the Express Application server.
- **Key Highlights:** Proactive Threat Modeling, Ingest Sanitisation, Decoupled Feeds.

SPEAKER NOTES:

As the Cybersecurity and Operations Lead for Critical Signal, my primary mission was to ensure that our platform is secure, resilient, and compliant by design.

When developing an aggregator, the single largest threat vector is the injection of malicious or malformed payloads from the external data feeds we poll. Instead of assuming public feeds like the NSW Rural Fire Service or Live Traffic are inherently safe, we treated them as 'untrusted' by default.

*Under our hybrid Agile-NIST approach, we established a strict **Trust Boundary** at the ingestion interface. Before any alert is parsed or saved, it must pass through a normalization and validation layer where HTML tags are stripped, dates are checked, and coordinates are verified. By isolating this parsing logic using the **Feed Adapter Pattern**, we ensure that a compromise or error in one external API cannot cascade to crash or compromise our entire ecosystem.*

Slide 2: Critical Vulnerability Mitigations (SQLi, CORS, & XSS)

Slide Visual Layout:

- Side-by-side comparison of **Vulnerable vs. Secure Code** highlighting parameterized SQL query structure in the DAO layer.
- Code snippet showing Express CORS origin restriction from `allowedOrigins.js`.
- **Key Highlights:** 100% SQLi Immunity, Strict Origin Whitelisting, React 19 Default Escaping.

SPEAKER NOTES:

To enforce our secure posture, we implemented three major technical controls in our prototype:

*First, we achieved **100% SQL Injection Immunity** by standardising all database interactions through our `BaseDAO.js` helper. Direct string concatenation is completely banned. As you can see on the slide, every query compiles with standard placeholder markers, passing inputs as bound parameters that SQLite executes safely.*

*Second, to protect our backend from unauthorized scraping or resource exploitation, we configured strict **CrossOrigin Resource Sharing (CORS)** middleware. In `allowedOrigins.js`, we whitelisted only our authenticated Next.js client domains. All other cross-origin REST requests are immediately rejected by the Express framework.*

*Finally, to mitigate **Cross-Site Scripting (XSS)**, we sanitize Alert description texts during ingestion, and our frontend is powered by **React 19**, which inherently escapes all variables rendered in JSX by default. If we must display dynamic HTML, we run a strict DOMpurify filter first.*

Slide 3: Operational Observability & Health Indicators

Slide Visual Layout:

- Screenshot of request logging middleware output from `logEvents.js`.
- Table displaying how `SourceHealthService.js` logs execution statistics (run counts, failures, records processed) in the database.

- **Key Highlights:** Asynchronous Auditing, Source Outage Resilience, Live Event Logging.

SPEAKER NOTES:

An application's security is only as strong as its visibility. In operations, we built a comprehensive auditing and health tracking system.

We developed a custom `logEvents.js` logging middleware. Every single API request is recorded asynchronously with a unique UUID, timestamp, method, and source origin, appending traces to `eventLog.txt`.

*Furthermore, in alignment with our Client's requirements, we implemented a dedicated **Source Health Tracking service**. Every time a scheduled cron job attempts to ingest alerts, our orchestrator logs the run's success or failure, processed record count, and precise error messages directly to the database. If NSW RFS goes down, our platform doesn't crash; the orchestrator catches the failure, writes the error log, and displays cached data to the user. This ensures operational availability during critical events, satisfying our reliability NFR.*

Slide 4: Data Privacy & Governance (Australian Privacy Act)

Slide Visual Layout:

- Visual mapping showing the decoupled user subscription schema.
- Highlights showing that zero Personally Identifiable Information (PII) is stored.
- **Key Highlights:** No physical address tracking, SHA-256 Hashing, Compliance by Minimisation.

PRADIP'S SPEAKER NOTES:

"Finally, let's address governance and compliance. The client brief outlines a subscription and notification model where users can receive alerts based on their region.

*To align with the **Australian Privacy Act 1988**, we implemented strict **Data Minimisation**. We made a conscious architectural decision never to collect user names, physical home addresses, or phone numbers.*

As our database schema shows, our `UserDAO.js` only records a standard email address, and user preferences are decoupled into a separate table storing only numeric category and region IDs. Furthermore, all authentication-related fields are hashed using cryptographic salting. This ensures that even in the highly unlikely event of a database compromise, zero sensitive personal details or geographic locations are exposed, keeping CSU staff and students completely safe."

PART 3: PROFESSIONAL Q&A PREPARATION

As the Cybersecurity & Operations Lead, anticipate targeted questions from the client panel. Use these evidencebased responses to project authority and technical competence:

Q1: Why did you use SQLite instead of MySQL when the Client Narrative Brief explicitly asked for MySQL in Session 1?

Answer: That was a deliberate design decision driven by feasibility and rapid prototyping. SQLite provided a zero-config, serverless database that allowed us to immediately establish the end-to-end relational data pipeline without complex server administration. However, because we built our database layers using the **Data Access Object (DAO) Pattern** and a fully relational schema matching MySQL structures, migrating the database from SQLite to MySQL in Session 2 is simple. We can plug in a MySQL driver and update our **db.js** file, and the rest of our query logic, services, and API routes will work without modification. This is an excellent architectural trade-off that expedited our Session 1 proof-of-concept while maintaining a clean, risk-free upgrade path.

Q2: What happens if an attacker compromises the NSW RFS feed itself and pushes a malicious script inside the description field of an alert?

Answer: Our architecture treats all external feed variables as hostile. During the ingestion lifecycle, the **IngestOrchestratorService** runs the feed description through a sanitisation filter that strips potential script tags or script-injection payloads. On the frontend, **React 19** automatically escapes all dynamic text elements rendered in the UI by default. This multi-layered defense guarantees that even if a feed provider is fully compromised, the malicious code is neutralized on our servers and can never execute in a user's browser.

Q3: How does the system handle rapid API rate limits or denial-of-service attempts on the backend?

Answer: For the Session 1 prototype, we secured our endpoints by placing them behind strict CORS origin whitelists, preventing unauthorized automated scraping bots from triggering queries. In addition, our data collection relies on background schedulers (**node-cron**) set to poll at a respectful 5-minute interval, rather than real-time polling on client page loads, which protects both our database and external agencies from rate limits. For the production deployment in Session 2, we will introduce a standard Express Rate Limiter middleware to restrict the maximum API requests per IP address, neutralizing denial-of-service risks.

Q4: How does the system ensure compliance with the Australian Privacy Act if users subscribe to receive specific alerts?

Answer: Our primary security control is **data minimisation**. We do not collect names, phone numbers, or physical home addresses. Users subscribe using only their email. The system maps their alert preferences to standardized, numeric region IDs (e.g., LGA or District labels) rather than precise geographic coordinates of their homes. This data structure ensures there is zero risk of tracking individual user locations, fully satisfying the privacy NFR.